Assignment No. -1

Problem Statement: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.

Learning Objectives: The objectives of this experiment are:

1. To learn the fundamentals of parallel programming using OpenMP.
2. To design and implement parallel BFS and DFS algorithms using OpenMP directives.
3. To analyze how parallelization improves performance compared to sequential algorithms.

Learning Outcome:

After completing this experiment, students will be able to:

1. Implement graph traversal techniques using trees or undirected graphs.
2. Apply OpenMP parallel constructs to improve algorithm efficiency.
3. Understand how parallelism reduces computation time in large graphs.

Software Requirements:

- **Operating System:** Windows / Linux / macOS (Linux preferred)
- **Programming Language:** C / C++
- **Compiler:** GCC with OpenMP support (`-fopenmp` flag)
- **Parallel Library:** OpenMP
- **IDE (Optional):** VS Code / Code::Blocks / Dev-C++
- **Hardware:** Multi-core processor (2+ cores), minimum 4 GB RAM
- **Graph Representation:** Adjacency List or Matrix

Theory:

Graph Traversal

Graph traversal refers to the process of visiting all vertices (nodes) of a graph in a systematic manner. Graphs are widely used in computer science applications such as network routing, social networks, search engines, and path finding.

Two of the most important traversal algorithms are:

- Breadth First Search (BFS)
- Depth First Search (DFS)

These algorithms can operate on trees or undirected graphs.

Breadth First Search (BFS)

Breadth First Search is a graph traversal technique in which nodes are explored level by level. It starts from a root node and first visits all its immediate neighbors before moving to the next level.

Characteristics of BFS

- Uses a queue data structure.
- Traverses nodes horizontally (level-wise).
- Guarantees the shortest path in an unweighted graph.
- Suitable for parallel processing because nodes at the same level can be processed simultaneously.

BFS Steps

1. Start from the source node.
2. Mark the node as visited.
3. Insert the node into a queue.
4. Remove a node from the queue.
5. Visit all unvisited adjacent nodes.
6. Mark them visited and add them to the queue.
7. Repeat until the queue becomes empty.

Parallel BFS using OpenMP

In parallel BFS:

- Multiple threads process neighboring nodes simultaneously.
- OpenMP directives like #pragma omp parallel for allow different threads to explore adjacency lists concurrently.
- Shared structures like visited arrays are used with synchronization to avoid race conditions.

Depth First Search (DFS)

Depth First Search is another graph traversal technique that explores nodes as deep as possible before backtracking.

Characteristics of DFS

- Uses a stack data structure or recursion.
- Traverses nodes depth-wise.
- Useful for cycle detection, topological sorting, and path finding.

DFS Steps

1. Start from the source node.
2. Mark it as visited.
3. Visit one unvisited adjacent node.
4. Continue exploring deeper nodes.
5. If no unvisited neighbor exists, backtrack.
6. Repeat until all nodes are visited.

Parallel DFS using OpenMP

Parallel DFS is more complex because DFS is inherently sequential. However, parallelism can be introduced by:

- Allowing multiple threads to explore different branches of the graph simultaneously.
- Using OpenMP tasks to process recursive calls in parallel.
- Ensuring proper synchronization for shared resources such as the visited array.

OpenMP in Parallel Programming

OpenMP (Open Multi-Processing) is a widely used API for shared memory parallel programming in C, C++, and Fortran.

Features of OpenMP

- Simple compiler directives
- Supports multithreading
- Efficient for loop parallelization
- Uses fork-join model

Basic OpenMP Directives

Example:

```
#pragma omp parallel for
for(int i = 0; i < n; i++)
{
  // parallel work
}
```

In this directive:

- The loop iterations are distributed among multiple threads.
- Each thread executes a portion of the work simultaneously.

Parallel BFS and DFS in Graphs

In large-scale graphs, sequential traversal becomes slow. Parallel algorithms improve performance by:

- Dividing work among threads
- Processing multiple nodes simultaneously
- Reducing execution time

Applications

Parallel BFS and DFS are used in:

- Web crawling
- Network analysis
- Artificial intelligence

- Social network analysis
- Shortest path computation

Using OpenMP helps achieve better utilization of multi-core processors.

Conclusion

In this experiment, the parallel implementation of Breadth First Search and Depth First Search using OpenMP was studied and implemented. BFS and DFS are fundamental graph traversal algorithms widely used in computer science applications. Overall, the study of parallel BFS and DFS provides valuable insights into graph processing and high-performance computing techniques.

Questions

1. What if BFS?
2. What is OpenMP? What is its significance in parallel programming?
3. Write down applications of Parallel BFS
4. How can BFS be parallelized using OpenMP? Describe the parallel BFS algorithm using OpenMP.
5. Write Down Commands used in OpenMP?
6. What if DFS?
7. Write a parallel Depth First Search (DFS) algorithm using OpenMP
8. What is the advantage of using parallel programming in DFS?
9. How can you parallelize a DFS algorithm using OpenMP?
10. What is a race condition in parallel programming, and how can it be avoided in OpenMP?

Assignment No. -2

Problem Statement: Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Learning Objectives:

1. To implement parallel versions of Bubble Sort and Merge Sort using OpenMP.
2. To compare the performance of sequential and parallel sorting algorithms.
3. To analyze how parallel execution improves computational efficiency.
4. To gain practical knowledge of OpenMP directives for parallel programming.

Learning Outcome:

After performing this experiment, students will be able to:

1. Apply OpenMP directives to parallelize sorting algorithms.
2. Measure and compare the execution time of sequential and parallel algorithms.
3. Understand how parallel processing improves performance on multi-core processors.
4. Implement and test parallel sorting algorithms using OpenMP.

Software Requirements:

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** C / C++
- **Compiler:** GCC with OpenMP support (-fopenmp)
- **Parallel Library:** OpenMP
- **IDE (Optional):** VS Code / Code::Blocks
- **Hardware:** Multi-core CPU (2+ cores), minimum 4 GB RAM

Theory:

Sorting Algorithms

Sorting is the process of **arranging elements of a list or array in a specific order**, typically ascending or descending. Sorting is a fundamental operation in computer science and is used in applications such as **database management, searching, data analysis, and optimization problems**. Sorting algorithms can be classified into two types:

- Sequential Sorting Algorithms – executed using a single processor.
- Parallel Sorting Algorithms – executed using multiple processors or threads simultaneously.

Parallel sorting algorithms help improve performance when working with large datasets.

Bubble Sort

Bubble Sort is one of the simplest sorting algorithms. It repeatedly compares adjacent elements and swaps them if they are in the wrong order. This process continues until the array becomes sorted.

Characteristics of Bubble Sort

- Simple and easy to implement.
- Uses comparison and swapping operations**.**
- Time complexity:
    - Best case: **O(n)**
    - Average case: **O(n²)**
    - Worst case: **O(n²)**
- Suitable mainly for small datasets**.**

Working Principle

1. Compare the first two elements of the array.
2. Swap them if they are in the wrong order.
3. Move to the next pair of elements.
4. Continue until the end of the array.
5. Repeat the process for the remaining elements until no swaps are required.

Parallel Bubble Sort

In parallel Bubble Sort:

- Multiple comparisons and swaps are performed simultaneously.
- OpenMP allows loops to run in parallel using directives such as:

```
#pragma omp parallel for
```

A common approach for parallel bubble sort is Odd-Even Transposition Sort, where:

- Odd indexed elements are compared in one phase.
- Even indexed elements are compared in another phase.
- These phases can be executed in parallel using multiple threads.

Merge Sort

Merge Sort is an efficient sorting algorithm based on the **Divide and Conquer technique**.

Characteristics of Merge Sort

- Divides the array into **smaller subarrays**.
- Sorts each subarray recursively.
- Merges sorted subarrays to produce the final sorted array.

Time Complexity

- Best Case: **O(n log n)**
- Average Case: **O(n log n)**
- Worst Case: **O(n log n)**

Working Principle

1. Divide the array into two halves.

2. Recursively sort each half.
3. Merge the two sorted halves into a single sorted array.
4. Continue until the entire array is sorted.

Parallel Merge Sort

Merge Sort is highly suitable for parallelization because:

- The **divide step** creates independent subproblems.
- These subproblems can be processed by **different threads simultaneously**.

Using OpenMP:

- Each recursive call can be assigned to a **separate thread**.
- OpenMP tasks or parallel sections allow multiple parts of the array to be sorted concurrently.

Example directive:

```
#pragma omp parallel sections
{
  #pragma omp section
  mergeSort(left half);

  #pragma omp section
  mergeSort(right half);
}
```

This allows both halves of the array to be sorted **at the same time**, improving performance.

OpenMP for Parallel Programming

OpenMP (Open Multi-Processing) is an **API that supports shared-memory parallel programming** in C, C++, and Fortran.

Features of OpenMP

- Easy to use **compiler directives**
- Supports **multi-threaded execution**
- Works on **multi-core processors**
- Uses **fork-join execution model**

Basic OpenMP Workflow

1. The main program runs sequentially.
2. When a parallel region is encountered, multiple threads are created.
3. The work is divided among threads.
4. After execution, the threads join back into the main thread**.**

Performance Measurement

Performance comparison between sequential and parallel algorithms is done using execution time measurement**.**

Execution time can be calculated using functions such as:

- omp_get_wtime() in OpenMP.

Example:

```
double start = omp_get_wtime();
/* algorithm execution */
double end = omp_get_wtime();
```

The difference between start and end time gives the total execution time**.**

Performance Analysis

When comparing sequential and parallel sorting algorithms:

- Sequential algorithms execute on one processor.
- Parallel algorithms use multiple threads simultaneously.
- Parallel algorithms often show reduced execution time, especially for large datasets.

However, the improvement depends on factors such as:

- Number of processors
- Size of data
- Overhead of thread creation

Conclusion

In this experiment, the implementation of Parallel Bubble Sort and Parallel Merge Sort using OpenMP was studied and analyzed. The sequential versions of Bubble Sort and Merge Sort were compared with their parallel implementations to evaluate performance improvements. Overall, this experiment demonstrates the advantages of parallel computing in improving algorithm efficiency and performance, and it highlights the importance of parallel programming techniques in modern high-performance computing systems.

Questions

1. How parallel Bubble sort work explain it with example
2. What are advantages to use bubble sort.
3. How parallel merge sort work explain with example
4. What are different metrics to measure the performance of merge sort?
5. Explain how parallel Merge Sort improves performance for large datasets**.**

Assignment No. -3

Problem Statement: Implement Min, Max, Sum and Average operations using Parallel Reduction.

Learning Objectives: The main objectives of this experiment are:

- To understand the concept of **parallel reduction** in parallel computing
- To learn how to compute **minimum, maximum, sum, and average** using parallel techniques
- To analyze the performance difference between **sequential and parallel approaches**
- To understand synchronization and memory considerations in parallel algorithms
- To gain hands-on experience in implementing reduction operations using parallel programming models

Learning Outcome: After completing this experiment, students will be able to:

- Explain the concept of reduction and its importance in parallel computing
- Implement parallel algorithms for **min, max, sum, and average**
- Apply reduction techniques using multi-threading or GPU-based programming
- Evaluate time complexity improvements using parallelism
- Identify challenges such as synchronization, race conditions, and load balancing

Software Requirements:

- Operating System: Windows / Linux / macOS
- Programming Language: C / C++ / Python
- Parallel Programming Tools (any one):
  - OpenMP (for CPU parallelism)
  - CUDA (for GPU programming)
  - MPI (for distributed systems)
- Compiler:
  - GCC (for C/C++)
  - NVCC (for CUDA programs)
- IDE (optional): Code::Blocks / Visual Studio / VS Code

Theory:

Introduction to Parallel Reduction

Parallel reduction is a technique used to combine elements of a dataset into a single result using an associative operation such as addition, minimum, or maximum. It is a key building block in parallel programming and is widely used in applications requiring aggregation of large datasets.

In traditional sequential programming, reduction operations iterate through the array one element at a time, resulting in **O(n)** time complexity. However, in parallel reduction, multiple elements are processed simultaneously, reducing the time complexity to approximately **O(log n)**.

Reduction Operations

Reduction operations take a collection of elements and reduce them to a single value:

- **Sum** → Addition of all elements
- **Minimum (Min)** → Smallest element in the array
- **Maximum (Max)** → Largest element in the array
- **Average** → Sum of elements divided by total number of elements

Mathematically:

Sum = Σ A[i]

Min = smallest(A[i])

Max = largest(A[i])

Average = Sum / N

Working of Parallel Reduction

Parallel reduction works in a **tree-like structure**, where elements are combined in pairs at each step:

Example:

Input: [a1, a2, a3, a4, a5, a6, a7, a8]

Step 1: [a1+a2, a3+a4, a5+a6, a7+a8]
Step 2: [(a1+a2)+(a3+a4), (a5+a6)+(a7+a8)]
Step 3: Final result

Each step halves the number of elements, leading to **log (n)** steps.

Algorithm for Parallel Reduction

1. Initialize input array
2. Divide array into blocks for parallel processing
3. Perform pairwise operations:

```
for stride = n/2 to 1:
   if thread index < stride:
      A[i] = operation(A[i], A[i + stride])
```

4. Synchronize threads after each iteration
5. Final result is stored in the first element

Implementation of Reduction Operations

Sum Reduction

- Each thread adds two elements
- Intermediate sums are combined until a final sum is obtained

Minimum Reduction

- Each thread compares two elements
- Stores the smaller value

Maximum Reduction

- Each thread compares two elements
- Stores the larger value

Average Calculation

- First compute sum using reduction
- Then divide by total number of elements

Key Concepts

1. Parallelism

Multiple computations are performed simultaneously using threads or processing units.

2. Synchronization

Threads must wait for others to complete before moving to the next stage to avoid incorrect results.

3. Memory Optimization

Using shared memory (in GPU programming) improves performance by reducing global memory access time.

4. Load Balancing

Ensures equal work distribution among threads to maximize efficiency.

Advantages

- Faster execution compared to sequential methods
- Efficient use of multi-core processors and GPUs
- Scalable for large datasets
- Reduced computational complexity

Limitations

- Requires synchronization overhead
- Performance depends on hardware capabilities
- Not efficient for small datasets
- Complexity in implementation

Applications

- Statistical analysis (mean, variance)
- Machine learning computations
- Image and signal processing
- Scientific simulations
- Big data analytics

Conclusion:

Parallel reduction is an essential technique for efficiently performing aggregation operations like **min, max, sum, and average** on large datasets. By dividing the workload among multiple processing units, it significantly reduces execution time. Understanding this concept helps in building optimized applications in high-performance computing environments.

Questions

1. What are the benefits of using parallel reduction for basic operations on large arrays?
2. How does OpenMP's "reduction" clause work in parallel reduction?
3. How do you set up a C++ program for parallel computation with OpenMP?
4. What are the performance characteristics of parallel reduction, and how do they vary based on input size?
5. How can you modify the provided code example for more complex operations using parallel reduction?

Assignment No. 4 -- Recurrent Neural Network (RNN)

Problem Statement: Use the **Google Stock Prices Dataset** and design a **time series analysis and prediction system using Recurrent Neural Network (RNN)**.

Learning Objectives:

- To understand the concept of Recurrent Neural Networks.
- To learn how RNN models handle sequential and time series data.
- To implement stock price prediction using deep learning techniques.
- To analyze trends and patterns in stock market data.

Learning Outcome:

After completing this experiment, students will be able to:

- Understand the concept of time series analysis.
- Implement RNN models for prediction of sequential data.
- Analyze and interpret stock price prediction results.

Software and Software Requirements:

Sr No	Software	Specification
1	Operating System	Windows / Linux
2	Programming Language	Python
3	Python Version	Python 3.8 or higher
4	IDE	Jupyter Notebook / Google Colab / VS Code
5	Deep Learning Framework	TensorFlow / Keras
6	Libraries	NumPy, Pandas, Matplotlib, Scikit-learn

Theory:

A **Recurrent Neural Network (RNN)** is a type of artificial neural network designed for processing **sequential data**. Unlike traditional neural networks, RNNs have feedback connections that allow information to persist over time.

RNNs are particularly useful for **time series analysis**, where past data influences future predictions.

Applications of RNN include:

- Stock price prediction
- Speech recognition
- Natural language processing
- Weather forecasting

The **Google Stock Prices Dataset** contains historical stock market data such as opening price, closing price, high, low, and trading volume. Using RNN, the model learns patterns from past stock prices and predicts future trends.

Algorithm:

1. Import necessary libraries such as NumPy, Pandas, Matplotlib, and TensorFlow/Keras.
2. Load the Google Stock Price dataset.
3. Perform data preprocessing and normalization.
4. Prepare the training and testing datasets.
5. Build the RNN model architecture.
6. Train the model using historical stock price data.
7. Predict stock prices using the trained model.
8. Plot predicted stock prices and compare them with actual prices.

Result:

The RNN model successfully analyzed historical stock price data and predicted future stock price trends with reasonable accuracy.

Conclusion:

This experiment demonstrated the implementation of **Recurrent Neural Network for time series prediction**. The RNN model effectively learned patterns from historical stock price data and generated predictions for future stock trends.

Oral Questions:

1. What is a Recurrent Neural Network (RNN)?
2. What is time series data?
3. What are the applications of RNN?
4. What is the difference between CNN and RNN?
5. What is the vanishing gradient problem?